Torneo de Komikan Concurrente

Tarea 2 – Sistemas Operativos

Estudiante: Benjamín Herrera

1. Diseño del esquema de sincronización

El programa está hecho en C++17 y utiliza hilos (`pthread`) y semáforos (`sem_t`). Recibe la cantidad de jugadores (N), el máximo de mesas (M) y un archivo de configuración opcional.

El hilo principal crea un hilo para cada cliente. Cada cliente avisa si corresponde al equipo de los pumas o al de los perros, usando los semáforos `sem_pumas` y `sem_perros`. El servidor espera una señal de cada equipo y luego pide una mesa usando `sem_mesas`. Cuando termina una partida, el juez libera la mesa.

Cada partida tiene un hilo juez. Este prepara el tablero y crea un hilo para el Puma y doce hilos para los perros. Las posiciones de las piezas y el estado de la partida se guardan en una estructura llamada Tablero. Cada tablero tiene un mutex (candado) para evitar que dos hilos cambien sus datos al mismo tiempo.

Para garantizar la exclusión mutua, cada hilo toma el mutex del tablero, revisa que la casilla destino esté libre, mueve la pieza y solo entonces suelta el mutex. Como la revisión y el movimiento ocurren dentro de la misma sección crítica, si dos piezas quieren la misma casilla, la primera que toma el mutex la ocupa y la segunda la encuentra ocupada y no se mueve.

También se usa el mutex `candado_pantalla` para que los mensajes de distintas partidas no se mezclen en la consola.

Los tiempos de espera de los movimientos se generan de forma aleatoria con `rand_r`. En el archivo de configuración se pueden indicar el tiempo mínimo y máximo, la semilla, la probabilidad de abandono y una posición inicial para el tablero.

2. Análisis de riesgos de concurrencia

Condiciones de carrera: pueden ocurrir si varios hilos intentan mover piezas al mismo tiempo. El mutex de cada tablero evita que dos hilos lo modifiquen a la vez. También se usa un mutex para ordenar los mensajes que aparecen en pantalla.

Deadlock: ocurre cuando un hilo queda esperando algo que otro no libera. En el programa, los mutex se liberan después de usarlos. Además, ningún hilo usa a la vez los mutex de dos tableros distintos, y el mutex de la pantalla siempre se toma después del mutex del tablero, nunca al revés. Por eso no se puede formar una espera circular. El servidor también revisa cada cierto tiempo si se pidió detener el programa.

Livelock: como las piezas esperan un tiempo aleatorio entre movimientos, es difícil que se bloqueen entre sí repitiendo siempre los mismos intentos. Además, cada partida tiene un máximo de 2000 movimientos y se declara empate, por lo que siempre termina.

Starvation: el programa no define un orden para que los hilos usen el tablero. Por eso, no se asegura que todos tengan las mismas oportunidades de realizar movimientos.

Abandono: los hilos revisan la probabilidad de abandono indicada en la configuración. Si un jugador abandona, la partida termina y gana el equipo contrario.

Interrupción con Ctrl+C: al presionar Ctrl+C se activa la variable `cortar`. El servidor deja de iniciar partidas y luego espera a que terminen los hilos antes de liberar los recursos utilizados.

3. Evaluación y decisiones de diseño

Se usa un mutex por cada tablero para que una partida no bloquee a las demás. Así pueden funcionar varias mesas al mismo tiempo, según el límite definido por `sem_mesas`. En cada mesa, solo un hilo puede modificar el tablero a la vez.

Los semáforos sirven para coordinar los equipos y controlar cuántas mesas están ocupadas. Los tiempos aleatorios hacen que los movimientos ocurran en distintos momentos. El juez crea los hilos, revisa el resultado y libera la mesa al terminar.

Se eligió un solo mutex por tablero porque es simple y seguro, y no permite deadlock. Su costo es que dentro de una mesa solo se mueve una pieza a la vez, pero casi no afecta el rendimiento: cada pieza espera entre 50 y 300 ms entre intentos y cada movimiento dura mucho menos, así que el mutex casi siempre está libre. Una alternativa era un mutex por casilla, que permite más concurrencia pero obliga a tomar varios mutex en un orden fijo para evitar deadlock; esa ganancia no se justifica aquí.

El programa usa los hilos, mutex y semáforos pedidos en la tarea. De todas formas, tiene algunos detalles a considerar. Para iniciar una partida se necesita una señal de cada equipo, pero el juez crea doce hilos para los perros. Esto significa que los hilos de clientes avisan que hay jugadores disponibles, pero no representan a cada perro por separado. Además, el código revisa movimientos hacia arriba, abajo, izquierda y derecha, por lo que no considera todas las conexiones diagonales que aparecen en el tablero del enunciado.